

VERIQTA



VERIQTA

VERIQTA

VERIQTA

VERIQTA



VERIQTA

VERIQTA

VERIQTA

VERIQTA

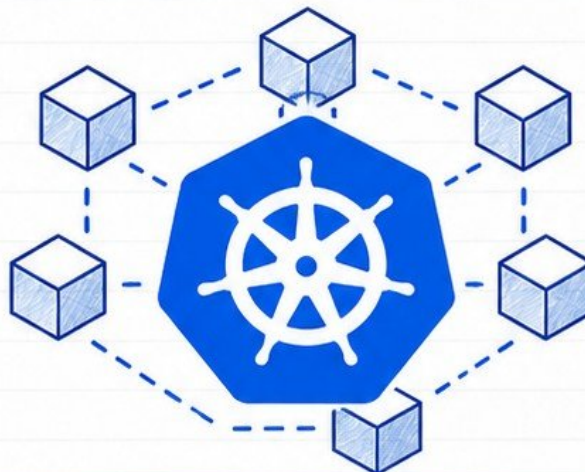
AZURE AKS HANDBOOK

From AKS Fundamentals to
Production Kubernetes Operations

VERIQTA

VERIQTA

- ✓ DESIGN
- ✓ DEPLOY
- ✓ SECURE
- ✓ SCALE
- ✓ OPERATE
- ✓ OBSERVE



- SECURE
- SCALABLE
- RELIABLE
- PRODUCTION READY



PRACTICAL INSIGHTS. REAL-WORLD SCENARIOS.
PRODUCTION-FOCUSED.

VERIQTA

VERIQTA



PRODUCTION
FOCUSED



BEST
PRACTICES



REAL-WORLD
EXAMPLES

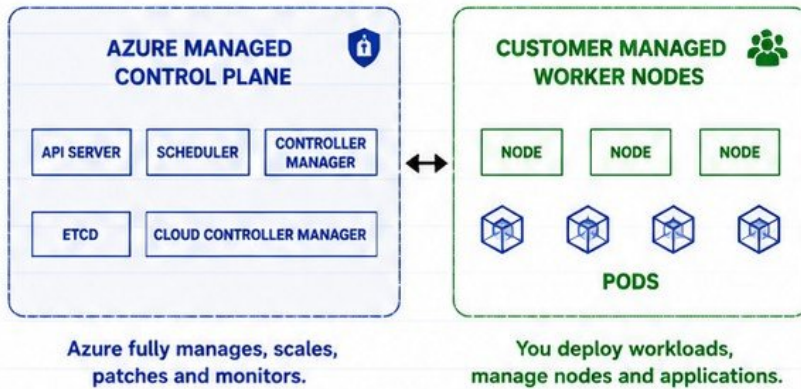


AZURE
INTEGRATION



AKS ARCHITECTURE UNDER THE HOOD

1. MANAGED CONTROL PLANE VS WORKER NODES



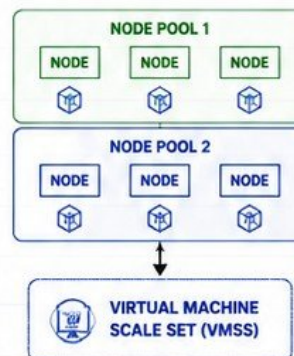
2. CONTROL PLANE COMPONENTS AND RESPONSIBILITIES

COMPONENT	RESPONSIBILITY
 API SERVER	Exposes Kubernetes API. Validates and processes all REST requests.
 SCHEDULER	Assigns Pods to nodes based on resources, policies and constraints.
 CONTROLLER MANAGER	Runs controller loops that maintain the desired state (Deployments, Nodes, etc.).
 ETCD	Consistent, distributed key-value store for all cluster data.
 CLOUD CONTROLLER MANAGER	Interacts with Azure APIs to manage cloud resources (Load Balancer, Routes, etc.).

3. AZURE-MANAGED VS CUSTOMER-MANAGED

AZURE MANAGED (YOU DON'T MANAGE)	CUSTOMER MANAGED (YOU MANAGE)
✓ Control plane components (API Server, Scheduler, etcd) ✓ Automatic upgrades and patching of control plane ✓ High availability of control plane ✓ Scaling of control plane ✓ Health monitoring of control plane	✓ Worker nodes and node pools ✓ OS updates on nodes ✓ Kubernetes workloads ✓ Applications and configurations ✓ Networking, RBAC, Policies, Secrets ✓ Monitoring, Logging and Backups

4. NODE POOLS & VM SCALE SETS



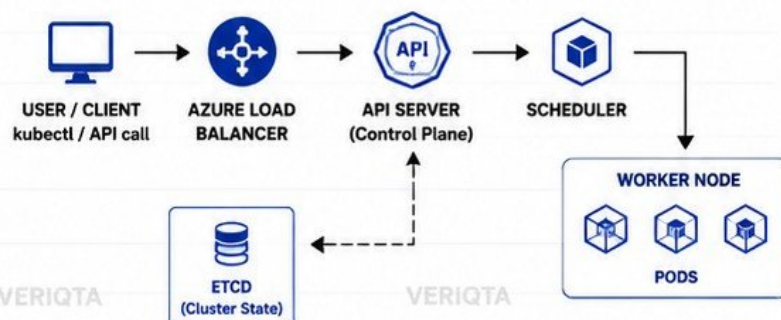
- Each node pool is built on Azure VM Scale Sets.
- Node pools can have different VM sizes, OS types and taints.
- Scale node pools independently.
- System node pool is created by default and runs critical system add-ons.

5. AKS RESOURCE GROUP ARCHITECTURE



AKS creates and manages required Azure resources inside the resource group.

6. KUBERNETES API REQUEST FLOW



PRODUCTION NOTE

Know exactly where Azure responsibility ends and yours begins. Secure, monitor and operate everything on the worker nodes and above.



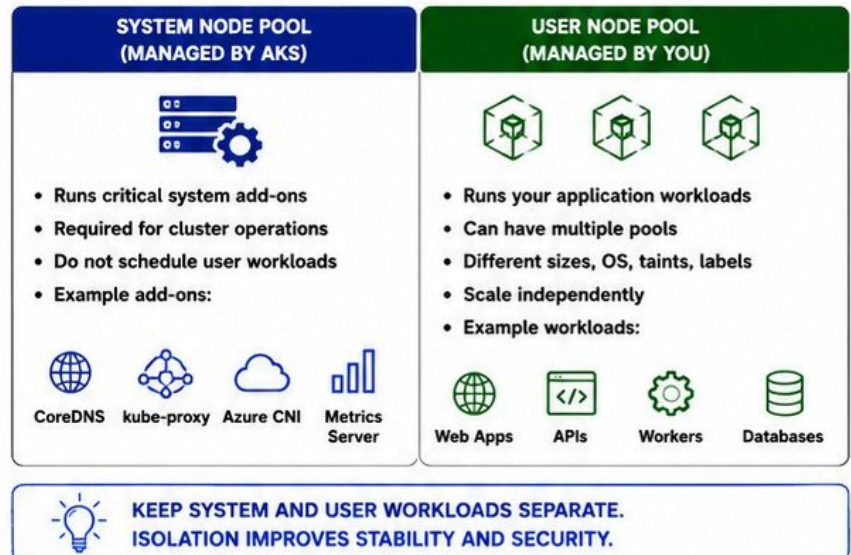
- Azure manages the control plane.
- You manage workloads, security, networking, identity and day-2 operations.
- Clarity here prevents outages and security gaps.

DESIGNING PRODUCTION AKS CLUSTERS

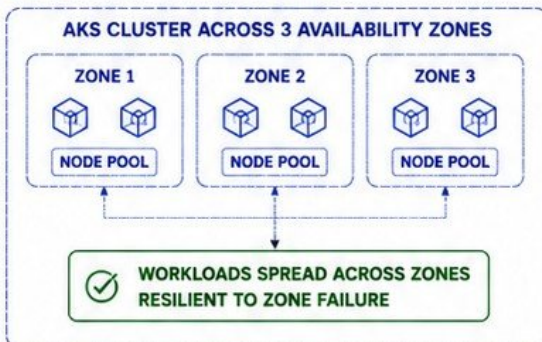
1. CLUSTER ARCHITECTURE DECISIONS

-  Define clear goals for the cluster (Reliability, Scalability, Security, Cost).
-  Choose the right Azure region close to your users and services.
-  Plan for high availability across Availability Zones.
-  Decide on networking model (Azure CNI recommended).
-  Determine workload placement and isolation strategy.
-  Plan for scalability and future growth.

2. SYSTEM VS USER NODE POOLS



3. AVAILABILITY ZONES



4. REGIONAL CONSIDERATIONS

-  Choose the right Azure region for latency and compliance.
-  Consider data residency and regulatory requirements.
-  Use paired regions for disaster recovery strategy.
-  Understand region pricing and service availability.

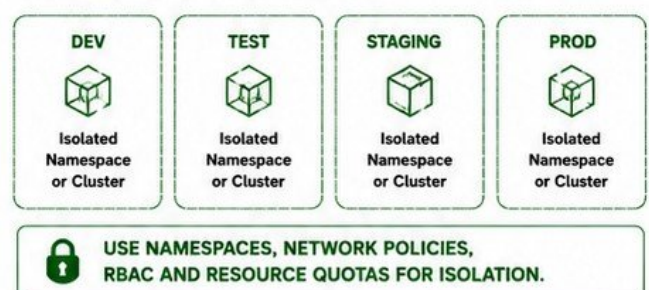
5. CLUSTER SIZING

-  Start with realistic workload requirements.
-  Choose correct VM sizes for compute needs.
-  Set requests and limits properly on workloads.
-  Enable cluster autoscaler for scale-out.
-  Right-size continuously to control costs.

6. MULTIPLE CLUSTERS VS SHARED CLUSTER

MULTIPLE CLUSTERS	VS	SHARED CLUSTER
- Stronger isolation - Independent upgrades - Different teams/environments - Higher cost - More operational overhead		- Lower cost - Simpler operations - Resource sharing - Risk of noisy neighbors - Complex isolation
BEST FOR: LARGE ORGANIZATIONS, STRICT ISOLATION, DIFFERENT LIFECYCLES		BEST FOR: SMALL TEAMS, NON-CRITICAL WORKLOADS

7. ENVIRONMENT ISOLATION



ENTERPRISE ADVICE

DESIGN FAILURE BOUNDARIES BEFORE WORKLOADS ARRIVE.

-  Plan for failure, not just for success.
-  Isolate workloads based on blast radius, not convenience.
-  Design for operations, upgrades, scaling and security from day one.
-  A well-designed cluster today prevents outages tomorrow.

AKS NETWORKING ARCHITECTURE

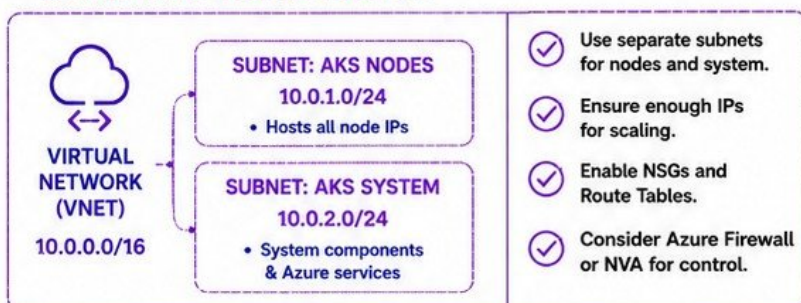
1. AZURE CNI NETWORKING OPTIONS

OPTION	DESCRIPTION
 AZURE CNI	- Each Pod gets an IP from the VNet - Best performance and scalability - Advanced networking features - Required for Network Policies
 AZURE CNI OVERLAY	- Pod IPs from overlay network - Node IPs from VNet - Supports larger Pod scale - Requires encapsulation (VXLAN)
 KUBENET	- Pod IPs from Kubernetes IP range - NAT (SNAT) used for outbound - Simpler but less scalable - Network Policies not supported

2. POD AND NODE ADDRESSING

POD ADDRESSING	NODE ADDRESSING
 - Unique IP for every Pod - IP allocated from VNet (Azure CNI) - Large IP space required	 - Nodes get IP from subnet (VNet) - Each node hosts multiple Pods

3. VNET AND SUBNET DESIGN



4. SERVICE CIDRS AND DNS

SERVICE CIDR (CLUSTER IP RANGE)	DNS SERVICE IP
 10.96.0.0/12 - Used for Kubernetes Services (ClusterIP) - Must not overlap with VNet or on-prem networks	 10.96.0.10 - CoreDNS service IP - Used for service discovery inside the cluster

5. ROUTE TABLES



- Associate Route Table with node subnet.
- Define routes for:
 - Internet (0.0.0.0/0)
 - On-prem networks
 - Azure services
 - Firewall / NVA

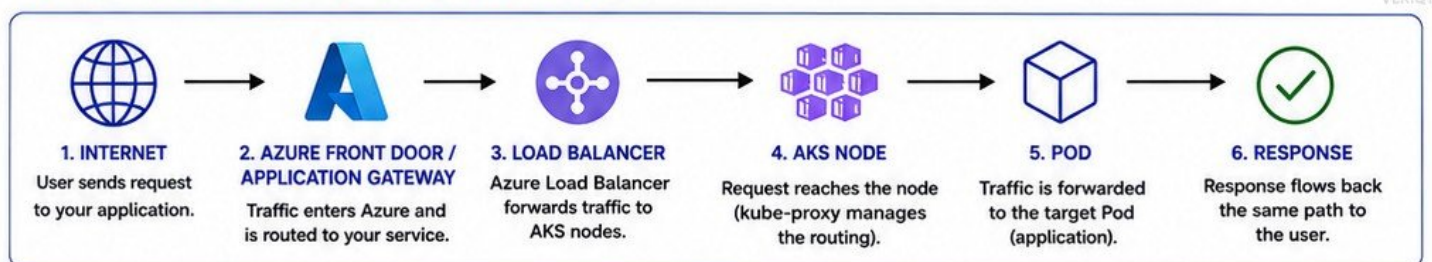
ROUTE TABLES CONTROL HOW TRAFFIC LEAVES OR ENTERS YOUR CLUSTER.

6. SNAT AND OUTBOUND CONNECTIVITY

OUTBOUND METHODS

-  **AZURE NAT GATEWAY**
 - Recommended for predictable outbound IP and scalability.
-  **AZURE FIREWALL**
 - Centralized outbound filtering and inspection.
-  **DEFAULT SNAT (KUBENET)**
 - Uses node IP for outbound (limited scale).

7. REQUEST FLOW: INTERNET → AZURE → AKS → POD



8. KUBENET CONSIDERATIONS

- Uses SNAT for outbound traffic (node IP).
- Limited to ~4000 Pods per node (port limits).
- Network Policies not supported.
- Simple to set up but less scalable.
- Not recommended for large production workloads.

9. PRODUCTION FAILURE: IP EXHAUSTION



- ✗ Insufficient IPs in subnet leads to Pods stuck in Pending.
- ✗ Node scale-out fails when no IPs are available.
- ✗ Services become unavailable during bursts.
- ✗ Difficult workloads to schedule and hard failures.

ALWAYS PLAN IP CAPACITY FOR 3X YOUR CURRENT PEAK. MONITOR IP USAGE AND SET ALERTS.



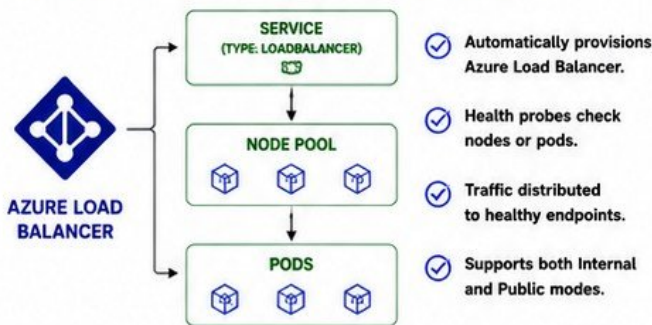
**DESIGN YOUR NETWORK FOR SCALE, SECURITY AND FAILURE.
A STRONG NETWORK LAYER IS THE FOUNDATION OF RELIABLE AKS OPERATIONS.**

SERVICES, LOAD BALANCING AND INGRESS

1. KUBERNETES SERVICE TYPES

TYPE	SCOPE	EXPOSED TO	USE CASE	PORT
CLUSTERIP 	Internal to cluster	Only other services and pods	Internal communication between microservices	Virtual IP (internal)
NODEPORT 	All nodes in the cluster	External via <NodeIP>:<Port>	Development or external access without cloud LB	30000-32767 (default range)
LOADBALANCER 	All nodes via cloud provider	External via cloud provider Load Balancer	Production workloads exposed externally	Assigned by cloud LB

2. AZURE LOAD BALANCER INTEGRATION



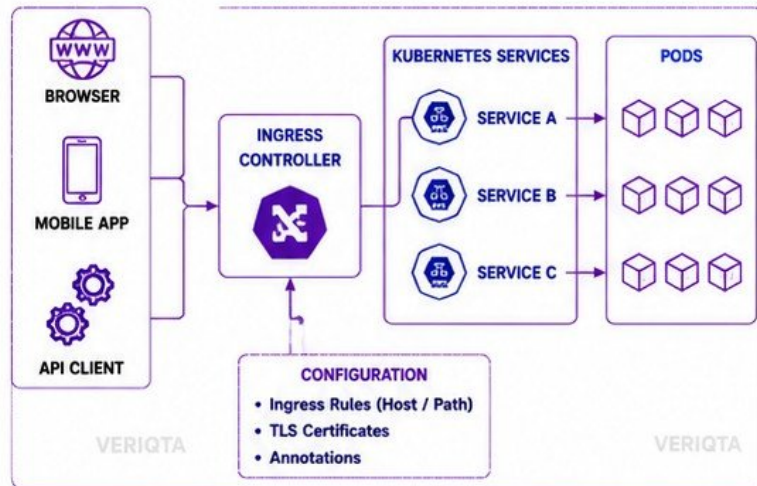
3. INTERNAL VS PUBLIC LOAD BALANCERS

INTERNAL LOAD BALANCER	PUBLIC LOAD BALANCER
 - Accessible only within VNet. - Private IP assigned. - Ideal for internal line-of-business applications.	 - Accessible from the internet. - Public IP assigned. - Use with NSGs, Azure Firewall, and WAF for protection.

7. TRAFFIC FLOW: CLIENT TO POD



4. INGRESS ARCHITECTURE



5. APPLICATION GATEWAY CONSIDERATIONS



- Layer 7 load balancer with WAF.
- SSL/TLS offload and re-encrypt.
- Path-based and host-based routing.
- Advanced features: Rewrite, Redirect, Cookie affinity.
- Use with Ingress Controller (AGIC) or standalone.

6. TLS TERMINATION OPTIONS

OPTION	TERMINATION POINT	DESCRIPTION	USE CASE
AT LOAD BALANCER / APP GW		TLS terminates at Azure Load Balancer or Application Gateway.	Centralized cert management, WAF, and offload.
AT INGRESS CONTROLLER		TLS terminates at Ingress Controller (Nginx/AGIC).	More control over routing and certs inside cluster.
END-TO-END (PASSTHROUGH)		TLS passes through LB and Ingress to the application.	Maximum security between client and app.



8. DEBUGGING INSIGHT

SERVICE EXISTS, BUT TRAFFIC NEVER REACHES THE WORKLOAD?

- Check Endpoints: `kubectl get endpoints <service>`
- Check Pods: `kubectl get pods -o wide`
- Check Events: `kubectl describe pod <pod-name>`
- Check Ingress Rules and Annotations
- Check Network Policies
- Check NSGs, Route Tables, and Firewall
- Verify Health Probes on Load Balancer / App GW



DESIGN SMART TRAFFIC PATHS. SECURE EVERY LAYER. OBSERVE EVERYTHING.
A WELL-DESIGNED SERVICE LAYER IS THE KEY TO RELIABLE APPLICATIONS.

AKS DNS AND SERVICE DISCOVERY

1. COREDNS IN AKS

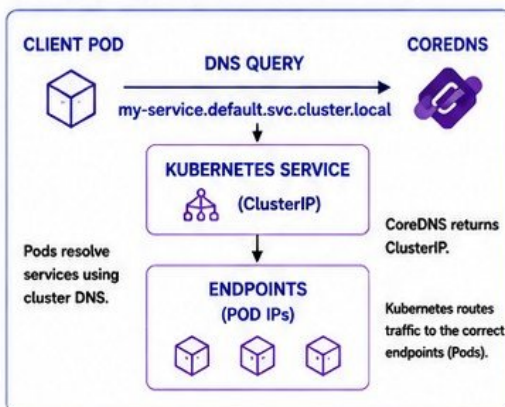


CoreDNS

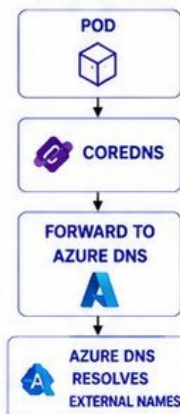
- CoreDNS is the default DNS server for every AKS cluster.
- Runs as Pods in kube-system namespace.
- Handles cluster DNS, service discovery and forwarding.
- Highly available (multiple replicas).
- Exposes DNS on ClusterIP (default: 10.96.0.10).

DEFAULT DNS SERVICE IP: 10.96.0.10
(Kubernetes DNS Service)

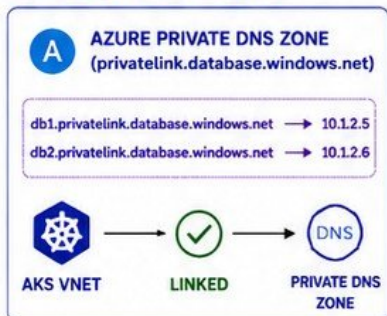
2. KUBERNETES SERVICE DISCOVERY



3. AZURE DNS INTERACTION



4. PRIVATE DNS ZONES



- Create Private DNS Zones for Azure Private Endpoints.
- Link the Private DNS Zone to the AKS VNet.
- Pods resolve private FQDNs to Private IPs automatically.
- Example: Access Azure SQL, Storage, Key Vault privately.

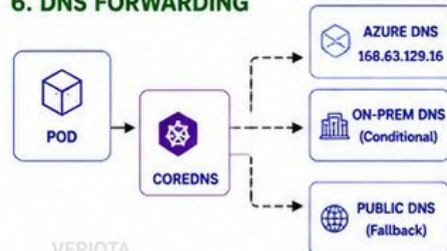
5. INTERNAL NAME RESOLUTION

NAME	RESOLVES TO	EXAMPLE
SERVICE (CLUSTERIP)	ClusterIP (Virtual IP)	<code>my-service.default.svc.cluster.local</code> → 10.96.x.x
POD (SAME NAMESPACE)	Pod IP	<code>my-pod.my-namespace.svc.cluster.local</code> → 10.244.x.x
POD (CROSS NAMESPACE)	Pod IP	<code>my-pod.other-ns.svc.cluster.local</code> → 10.244.x.x
EXTERNAL NAME	External FQDN	<code>api.contoso.com</code> → 52.x.x.x



ALWAYS USE FQDN INSIDE THE CLUSTER:
`<service>.<namespace>.svc.cluster.local`

6. DNS FORWARDING



- CoreDNS forwards queries based on zones and rules.
- Configure conditional forwarding for on-prem domains.
- Use Azure DNS for public internet names.
- Fallback ensures name resolution always works.

7. COREDNS CUSTOMIZATION

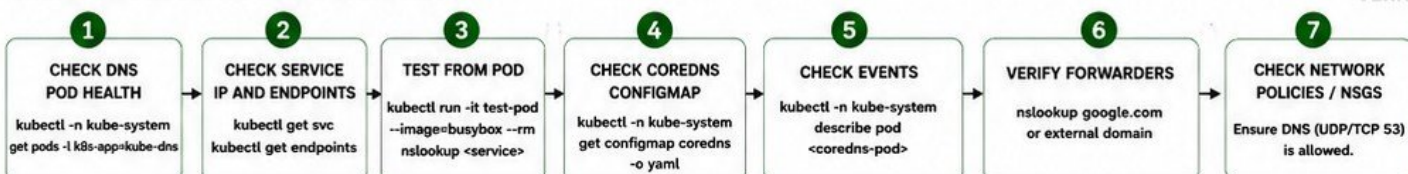
```

CUSTOM COREDNS CONFIG
.:53 {
  errors
  health
  ready
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
  }
  forward . 168.63.129.16 10.0.0.10
  cache 30
  loop
  reload
  loadbalance
}

```

- ✓ Edit CoreDNS ConfigMap in kube-system namespace.
- ✓ Add custom forwarders and stub domains.
- ✓ Tune cache, timeouts and load balancing.
- ✓ Keep changes under version control.

8. DNS TROUBLESHOOTING WORKFLOW



9. PRODUCTION FAILURE: PODS INTERMITTENTLY FAIL TO RESOLVE DEPENDENCIES



COMMON CAUSES

- ✗ CoreDNS Pods crash or restart.
- ✗ Node DNS settings overridden.
- ✗ Network Policies blocking DNS.
- ✗ Azure DNS or on-prem DNS issues.
- ✗ High DNS latency or packet loss.

IMPACT

- Pods cannot reach databases, APIs or external services.
- Applications show timeout errors.
- Intermittent failures under load.
- Deployment rollouts fail readiness.

HOW TO PREVENT

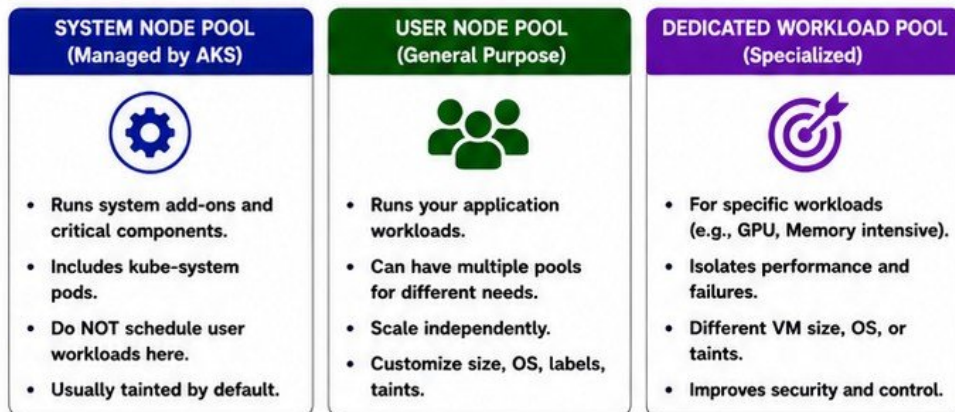
- ✓ Monitor CoreDNS health and latency.
- ✓ Use multiple replicas for CoreDNS.
- ✓ Allow DNS traffic in Network Policies.
- ✓ Use Private DNS Zones for critical services.
- ✓ Implement DNS SLOs and alerting.



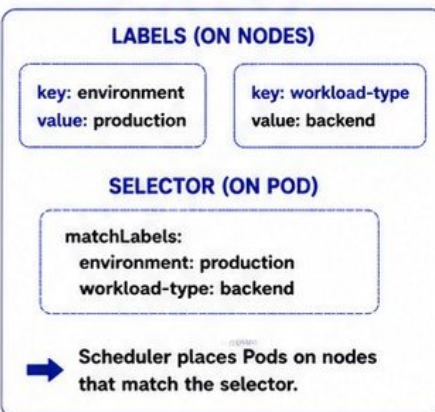
DNS IS THE FOUNDATION OF COMMUNICATION IN KUBERNETES.
IF DNS IS SLOW OR BROKEN, EVERYTHING IN THE CLUSTER FAILS.

NODE POOLS AND WORKLOAD PLACEMENT

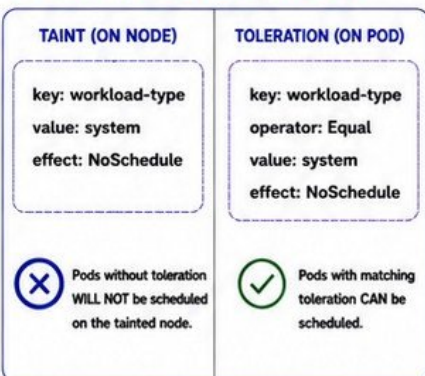
1. NODE POOL TYPES



2. LABELS AND SELECTORS



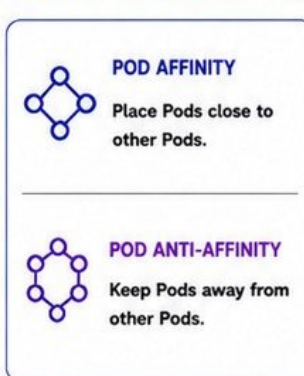
3. TAINTS AND TOLERATIONS



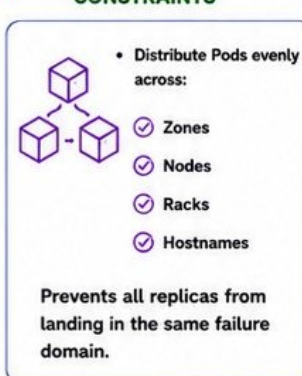
4. NODE AFFINITY



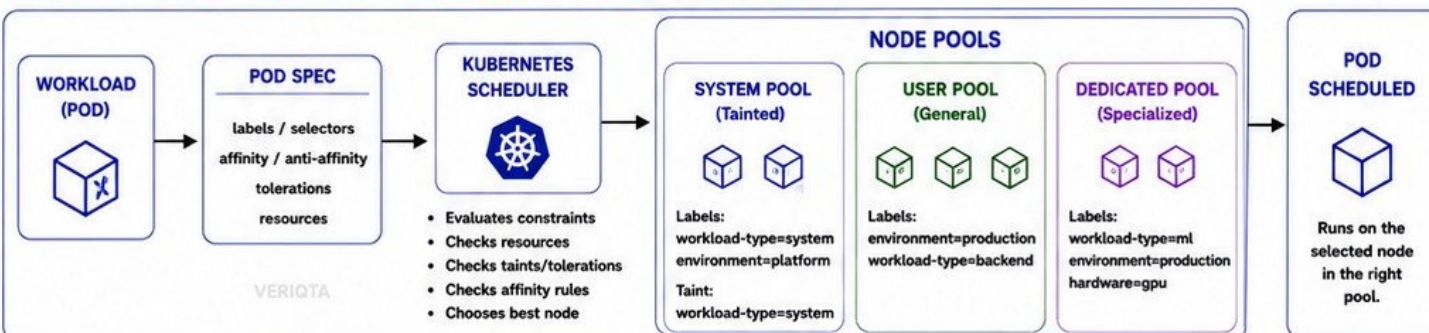
5. POD AFFINITY AND ANTI-AFFINITY



6. TOPOLOGY SPREAD CONSTRAINTS



7. ARCHITECTURE: WORKLOAD → SCHEDULER → CORRECT NODE POOL



8. KEY BEST PRACTICES

- ✓ Keep system workloads on system node pool.
- ✓ Use dedicated pools for different workload types.
- ✓ Use labels and selectors for placement control.
- ✓ Use taints and tolerations to enforce isolation.
- ✓ Use affinity/anti-affinity for placement strategy.
- ✓ Use topology spread to improve availability.
- ✓ Right-size node pools and enable autoscaling.



9. SENIOR ENGINEER TIP

SEPARATE WORKLOADS BY OPERATIONAL REQUIREMENTS, NOT RANDOMLY.

- ✓ Different teams? Use different pools.
- ✓ Different performance needs? Use dedicated pools.
- ✓ Different security levels? Isolate them.
- ✓ Different upgrade cadence? Separate them.
- ✓ Failure isolation starts with smart placement.



GOOD PLACEMENT STRATEGY = PERFORMANCE, RELIABILITY, SECURITY AND SCALE.
POOR PLACEMENT STRATEGY = INCIDENTS, NOISE AND HIGHER COSTS.

AKS IDENTITY AND MICROSOFT ENTRA ID

1 MICROSOFT ENTRA INTEGRATION

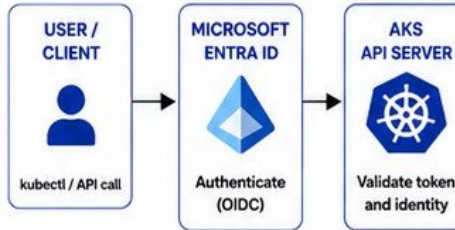


- Central identity provider
- Single sign-on to AKS
- Supports users, groups and service principals
- Integrated with Azure AD (Entra ID)

VERIQTA

VERIQTA

2 KUBERNETES AUTHENTICATION



Authentication answers: Who are you?

3 KUBERNETES RBAC



Roles define permissions



RoleBindings assign roles



ClusterRoles for cluster-wide access



Namespace isolation



Applies inside Kubernetes



Authorization answers: What can you do in Kubernetes?

4 AZURE RBAC FOR KUBERNETES

SCOPE	EXAMPLE ROLES
Azure Subscription	Contributor, Reader, Owner
Resource Group	Contributor, Reader
AKS Cluster	Azure Kubernetes Service RBAC roles



Controls access to Azure resources used by Kubernetes (not objects inside it).

5 MANAGED IDENTITIES

VERIQTA



SYSTEM ASSIGNED IDENTITY

- Tied to a single AKS resource
- Managed by Azure
- Used by the cluster or node pool



USER ASSIGNED IDENTITY

- Reusable across resources
- Attach to nodes, pods, or applications
- More control and flexibility

Use managed identities to securely access Azure services without secrets.

6 WORKLOAD IDENTITY

VERIQTA



7 SERVICE ACCOUNTS

VERIQTA



- Identity for workloads in AKS
- Used by Pods to access Kubernetes API
- Can be linked with Workload Identity for Azure access
- Avoid storing secrets in Pods

VERIQTA

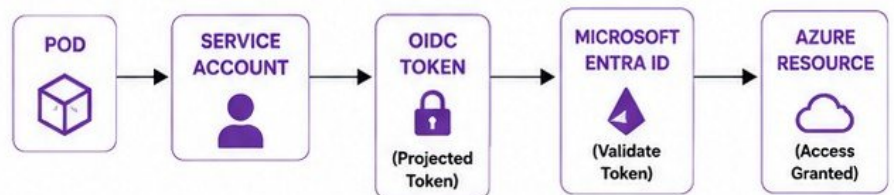
VERIQTA

8 TOKEN FLOW (WORKLOAD IDENTITY)

VERIQTA

VERIQTA

VERIQTA



Short-lived tokens are automatically rotated and securely validated.

VERIQTA

9 AUTHENTICATION VS AUTHORIZATION



AUTHENTICATION

Verifies the identity of the user or workload.

VS



AUTHORIZATION

Determines what the identity is allowed to do.

Example:
Microsoft Entra ID validates your token.

Example:
Kubernetes RBAC or Azure RBAC controls permissions.



You must pass both to successfully access resources.

10 SECURITY REMINDER



AVOID DISTRIBUTING LONG-LIVED AZURE CREDENTIALS TO WORKLOADS



NO SERVICE PRINCIPAL KEYS



NO CONNECTION STRINGS



NO SECRET FILES



NO STATIC TOKENS



Use Managed Identities and Workload Identity. Keep workloads secret-free and secure.

SECRETS AND CONFIGURATION

1 CONFIGMAPS



VERIQTA

- Store non-sensitive configuration data as key-value pairs
- Used by Pods as environment variables or mounted files
- Ideal for application settings, feature flags, config files

VERIQTA

2 KUBERNETES SECRETS



VERIQTA

- Store sensitive data (username, password, token, TLS, API keys)
- Base64 encoded (not encrypted)
- Use as environment variables or mounted as files
- Limit access with RBAC

VERIQTA

3 AZURE KEY VAULT



VERIQTA

- Centralized, cloud-native secrets management
- Encrypts secrets at rest and in transit
- Fine-grained access policies and auditing
- Secret versions and rotation
- Integrates with AKS via managed identity

4 SECRETS STORE CSI DRIVER



VERIQTA

- Mounts secrets from external stores (Key Vault) directly into Pods
- No storing secrets in Kubernetes etcd
- Auto-refresh secrets without Pod restart (optional)
- Uses gRPC CSI interface

VERIQTA

5 WORKLOAD IDENTITY INTEGRATION



- Pod uses a Kubernetes Service Account
- AKS federates the identity with Microsoft Entra ID (OIDC)
- Entra ID issues a token to access Azure Key Vault
- No secrets or credentials stored in the Pod

VERIQTA

6 SECRET ROTATION



- Rotate secrets regularly in Azure Key Vault
- New versions are available automatically
- CSI Driver syncs updates to mounted secrets
- Applications pick up changes without restart (if re-read supported)

VERIQTA

7 ENVIRONMENT VARIABLES VS MOUNTED SECRETS

ENVIRONMENT VARIABLES

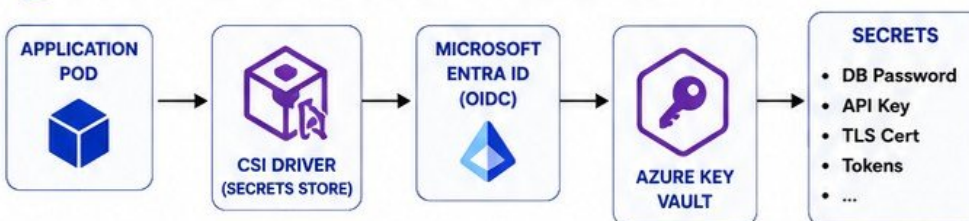
- ✓ Easy to use
- ✓ Good for simple secrets
- ✗ Visible in Pod spec (base64)
- ✗ Requires Pod restart to update
- ✗ Higher exposure in memory

MOUNTED SECRETS (FILES)

- ✓ More secure
- ✓ Supports auto-rotation
- ✓ Large secrets supported
- ✓ Not exposed in env
- ✓ No Pod restart needed (if app reloads)

VERIQTA

8 PRODUCTION SECRET-MANAGEMENT ARCHITECTURE



VERIQTA

KEY BENEFITS

- ✓ No secrets in Kubernetes etcd
- ✓ Centralized security and auditing
- ✓ Fine-grained access control
- ✓ Automatic rotation and updates
- ✓ Works across environments

IDENTITY + ACCESS CONTROL LAYER

RBAC
(KUBERNETES)AZURE RBAC
(KEY VAULT)MANAGED IDENTITY /
WORKLOAD IDENTITY

VERIQTA

VERIQTA

VERIQTA

BEST PRACTICES

- ✓ Use Workload Identity, not secrets
- ✓ Use Secrets Store CSI Driver
- ✓ Rotate secrets regularly
- ✓ Use least-privilege access
- ✓ Audit and monitor secret access



SECURITY REMINDER

KUBERNETES SECRETS ALONE ARE NOT A COMPLETE SECRETS STRATEGY



Stored in etcd
(base64 encoded)



No native encryption
at rest



Harder to rotate
safely



Higher risk of
exposure

SECURE SECRETS. SECURE WORKLOADS. SECURE EVERYTHING.

AKS STORAGE ARCHITECTURE

1 KUBERNETES STORAGE OBJECTS



PERSISTENTVOLUME (PV)

A piece of storage in the cluster provisioned by admin or dynamically.



PERSISTENTVOLUMECLAIM (PVC)

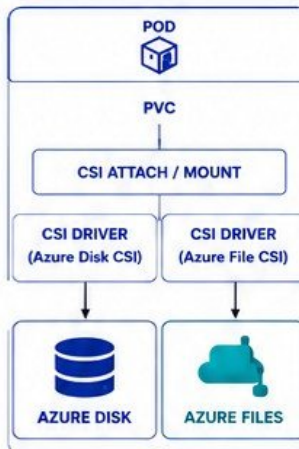
A request for storage by a user. Pods consume storage through PVCs.



STORAGECLASS

Defines the type of storage and how it should be provisioned (dynamic).

2 CSI ARCHITECTURE



- CSI (Container Storage Interface) is the standard for storage integration.
- CSI driver exposes storage capabilities to Kubernetes.
- Supports dynamic provisioning, attach, mount and snapshot.
- Works with all major Azure storage services.

3 AZURE STORAGE OPTIONS



AZURE DISK

- Block storage for VMs and containers
- Premium SSD / Standard SSD



AZURE FILES

- Fully managed file shares in the cloud
- SMB / NFS protocols



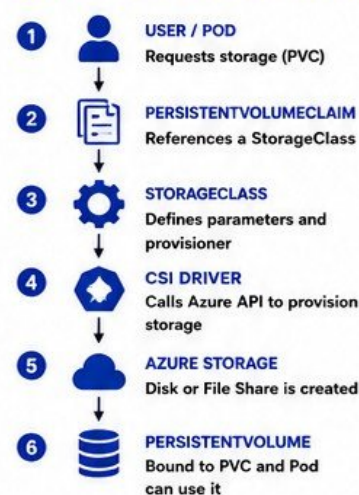
MANAGED DISKS

- Underlying storage for Azure Disks
- High availability, durable and performant

4 ACCESS MODES

ACCESS MODE	DESCRIPTION
ReadWriteOnce (RWO) 	Volume can be mounted as read-write by a single node.
ReadOnlyMany (ROX) 	Volume can be mounted as read-only by many nodes.
ReadWriteMany (RWX) 	Volume can be mounted as read-write by many nodes.
ReadWriteOncePod (RWOP) 	Volume can be mounted as read-write by a single Pod.

5 DYNAMIC PROVISIONING FLOW



6 STATEFUL WORKLOADS

AKS storage is critical for stateful applications such as:

 DATABASES (PostgreSQL, MySQL)	 MESSAGING (RabbitMQ, Kafka)	 SEARCH (Elasticsearch)
 FILE SERVERS (Nextcloud, SMB)	 MONITORING (Prometheus)	 AI / ML Workloads

- ✓ Use RWO or RWX based on workload needs.
- ✓ Ensure high availability with replication.
- ✓ Place Pods in the same zone as the volume when required.

7 AZURE DISK vs AZURE FILES – DECISION TABLE

FEATURE	AZURE DISK	AZURE FILES
STORAGE TYPE	Block Storage	File Storage
ACCESS PROTOCOL	Raw block (ext4, xfs)	SMB / NFS
ACCESS MODES	RWO, ROX, RWX (Preview)	RWX
PERFORMANCE	High IOPS, Low latency	Good throughput, higher latency
SCALABILITY	Scale by size (up to 32 TiB)	Scale by share capacity (up to 100 TiB)
USE CASES	Databases, High performance apps, Transactional workloads	File sharing, Content management, Lift-and-shift applications
BACKUP & SNAPSHOT	Snapshots supported	Snapshots supported
COST	Higher for Premium performance	Lower cost for large capacity
BEST FOR	Stateful apps needing high IOPS and low latency	Shared access, Lift-and-shift, File-based workloads

8 PRODUCTION CONSIDERATIONS

- 📍 Storage topology affects scheduling: Pods can only run on nodes that can access the volume.
- 🛡️ Use zone-redundant storage for high availability.
- 📊 Match StorageClass to workload performance needs.
- 📈 Monitor disk IOPS, latency, throughput and capacity.
- 💾 Plan for backups and DR (snapshots, replication).
- 🕒 Test restore process regularly.
- ⚠️ Avoid hostPath for production workloads.



PRODUCTION NOTE

STORAGE TOPOLOGY CAN DETERMINE WHERE A POD CAN ACTUALLY RUN.
DESIGN STORAGE WITH YOUR WORKLOAD ARCHITECTURE IN MIND.

- ✓ Know your access modes.
- ✓ Choose the right storage for the right workload.
- ✓ Plan for performance, availability and recovery.

SCALING AKS

1 HORIZONTAL POD AUTOSCALER (HPA)



- Scales the number of Pods based on metrics.
- Common metrics: CPU, Memory, Custom, External
- Checks metrics every 15 seconds by default.
- Requires resource requests to work correctly.

VERIQTA

Example: Scale from 3 to 10 Pods when CPU > 70%

2 VERTICAL POD AUTOSCALER (VPA) CONSIDERATIONS



VERIQTA

- Adjusts CPU/Memory requests and limits for Pods.
- Works in "Recommendation" or "Auto" mode.
- Not a replacement for HPA.
- Best for rightsizing workloads.

Use VPA to set the right size.
Use HPA to scale the right number.

3 CLUSTER AUTOSCALER



VERIQTA

- Automatically adjusts the number of nodes in the cluster.
- Adds nodes when Pods are pending.
- Removes nodes when underutilized.
- Requires proper node pools and permissions.

Ensures the cluster has enough capacity for your Pods.

4 NODE AUTOSCALING

VERIQTA



VERIQTA

- Enabled on agent node pools.
- Configurable min, max and initial node counts.
- Scale up uses Cluster Autoscaler.
- Scale down removes empty or underutilized nodes.

Example: Min 3 nodes | Max 20 nodes
Scale based on demand.

5 KEDA (KUBERNETES-BASED EVENT DRIVEN AUTOSCALING)



KEDA

- Scales Pods from 0 to N based on external events.
- Supports 50+ scalers (Queue, Topic, DB, HTTP, Cron, etc.)
- Ideal for cost-efficient workloads.
- Works with HPA.

Example: Scale from 0 to 20 Pods based on messages in Service Bus queue.

6 REQUESTS AND LIMITS

VERIQTA

✓ REQUESTS

- Guaranteed minimum resources.
- Used by Scheduler for placement.

✓ LIMITS

- Maximum resources a container can use.
- Prevents noisy neighbor issues.

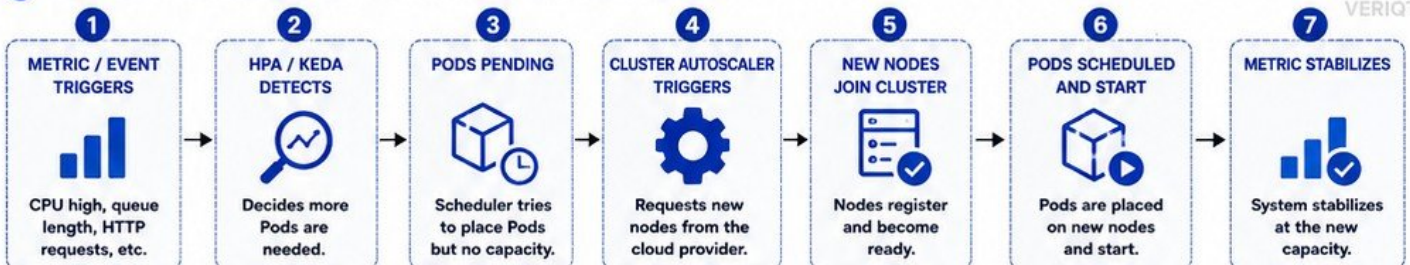
BEST PRACTICE

- ✓ Always set requests and limits.
- ✓ Requests < Limits
- ✓ Start with real usage metrics.
- ✓ Revisit regularly.

7 SCALE-OUT SEQUENCE (HOW EVERYTHING WORKS TOGETHER)

VERIQTA

VERIQTA



ALL COMPONENTS MUST WORK TOGETHER FOR SUCCESSFUL SCALING.

HPA / KEDA → CLUSTER AUTOSCALER → NODE POOL → POD SCHEDULER

VERIQTA

8 SCALING BOTTLENECKS

VERIQTA

VERIQTA

- ⚠️ **CPU / MEMORY LIMITS**
Pods hit limits and cannot scale effectively.
- ⚠️ **INSUFFICIENT NODES**
Cluster Autoscaler cannot add nodes (max reached, quota limits, or misconfiguration).
- ⚠️ **POOR REQUESTS/LIMITS**
Incorrect requests lead to pending Pods or poor resource utilization.
- ⚠️ **STORAGE LIMITS**
Disk IOPS, throughput or storage capacity become a bottleneck.

VERIQTA

- ⚠️ **NETWORK BOTTLENECKS**
Bandwidth, SNAT ports or latency limits affect scaling.
- ⚠️ **EXTERNAL DEPENDENCIES**
Databases, APIs or queues cannot handle increased load.
- ⚠️ **APPLICATION LIMITS**
App configuration or connection limits block scaling.

9 BEST PRACTICES FOR SCALING

VERIQTA

- ✓ Design for scale from day one.
- ✓ Set accurate resource requests and limits.
- ✓ Use HPA for steady-state scaling.
- ✓ Use KEDA for event-driven scaling (scale-to-zero).
- ✓ Enable Cluster Autoscaler with well-configured node pools.
- ✓ Use multiple node pools for different workloads.
- ✓ Monitor metrics, logs and events continuously.
- ✓ Test scaling during load tests.

VERIQTA



SCALING NOTE

POD SCALING MEANS LITTLE IF THE INFRASTRUCTURE UNDERNEATH CANNOT SCALE.
SCALE APPLICATIONS, NODES, AND INFRASTRUCTURE TOGETHER.

- ✓ Plan capacity.
- ✓ Test scaling.
- ✓ Monitor limits.

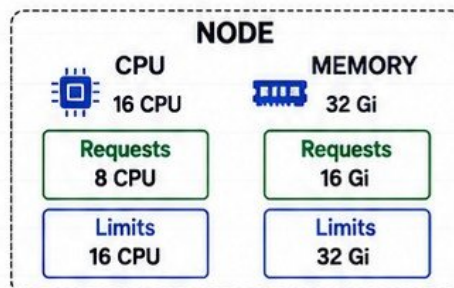
- ✓ Remove bottlenecks.
- ✓ Automate scaling.
- ✓ Review regularly.

11. RESOURCE MANAGEMENT AND SCHEDULING

- CPU requests and limits
- Memory requests and limits
- QoS classes
- ResourceQuota
- LimitRange
- Namespace isolation
- Scheduler decisions
- CPU throttling
- Memory pressure
- OOMKilled

1. REQUESTS VS LIMITS

TYPE	PURPOSE
REQUESTS	Minimum resources the container is guaranteed to get
LIMITS	Maximum resources the container is allowed to use



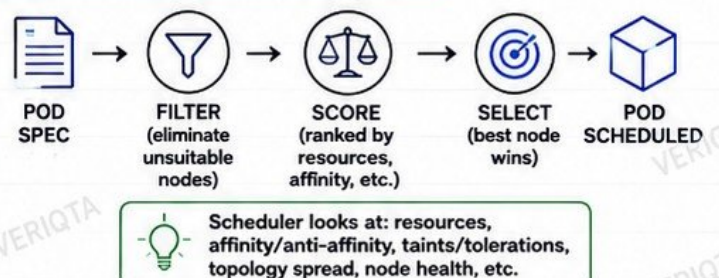
KEY IDEA

Scheduler uses **REQUESTS** to place pods. Limits control how much a container can actually use at runtime.

2. QOS CLASSES

QOS CLASS	CONDITION	BEHAVIOR
GUARANTEED	Requests = Limits for CPU and Memory	Highest priority. Least likely to be killed.
BURSTABLE	Requests < Limits for at least one resource	Medium priority. Can burst up to the limit.
BESTEFFORT	No requests No limits	Lowest priority. First to be killed under pressure.

3. SCHEDULER DECISION FLOW



4. RESOURCE QUOTA



NAMESPACE

QUOTA EXAMPLE

- CPU Requests: 10 CPU
- Memory Requests: 20Gi
- Limits: 20 CPU / 40Gi
- Pods: 100
- Services: 20

Prevents a namespace from consuming more than its allowed quota.

5. LIMITRANGE

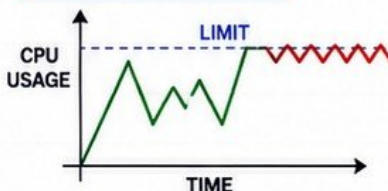


DEFAULTS & BOUNDARIES

- Default requests/limits
- Min / Max values
- Ensures every container has reasonable resources

Protects the cluster from containers with no or extreme resource values.

6. CPU THROTTLING



When CPU usage exceeds the limit, Kubernetes throttles the container. Result: Slow performance and high latency.

7. MEMORY PRESSURE & OOMKILLED



When memory exceeds available limits, the kernel OOM Killer terminates the victim container.

PRODUCTION FAILURE: HEALTHY APPLICATION REPEATEDLY DIES UNDER LOAD



COMMON ROOT CAUSES

- Memory limit too low → OOMKilled
- CPU limit too low → Throttling → Timeouts
- Missing requests → Evicted under pressure
- Incorrect HPA metrics
- No headroom for spikes



HOW TO DEBUG

1. kubectl describe pod <pod>
2. Check Events for OOMKilled / Evicted
3. Review Metrics (CPU / Memory)
4. Check Limits, Requests, Quota, and LimitsRange
5. Validate HPA and dependency latency



BEST PRACTICE

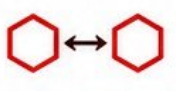
- Set requests and limits properly
- Leave headroom (20–30%)
- Monitor and alert early
- Test under real production load

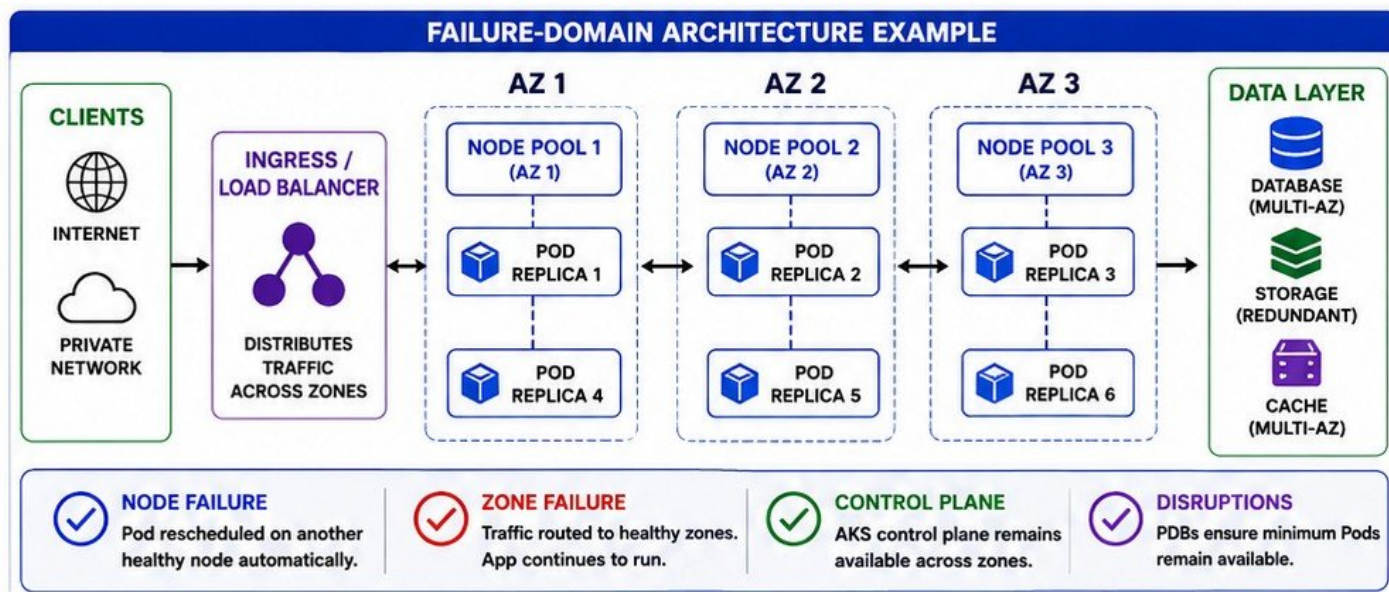


SENIOR ENGINEER TIP

Always design for the worst day, not the average day. Resources are cheap. Outages are expensive.

12. HIGH AVAILABILITY AND RESILIENCE

 AVAILABILITY ZONES Distribute workloads across AZs to survive datacenter failures.	 POD REPLICAS Run multiple replicas so the app stays available if a Pod fails.	 POD DISRUPTION BUDGETS Limits how many Pods can be disrupted at the same time.	 TOPOLOGY SPREAD Evenly spread Pods across nodes and zones.	 ANTI-AFFINITY Prevent Pods from running on the same node or zone.
 NODE FAILURES Workloads reschedule automatically to healthy nodes.	 ZONE FAILURES Apps continue running when an entire zone goes down.	 CONTROL-PLANE AVAILABILITY AKS control plane is multi-zone and highly available.	 GRACEFUL DISRUPTION Drain nodes safely without impacting application uptime.	 RESILIENCE GOAL Design for failure. Recover automatically. Keep serving users.



BEST PRACTICES FOR HIGH AVAILABILITY

 USE AT LEAST 3 AVAILABILITY ZONES	 RUN MULTIPLE REPLICAS	 SET POD DISRUPTION BUDGETS	 USE TOPOLOGY SPREAD CONSTRAINTS	 APPLY ANTI-AFFINITY RULES	 MONITOR AND ALERT EARLY	 TEST FAILURE SCENARIOS REGULARLY
--	--	---	--	---	--	---


RELIABILITY NOTE

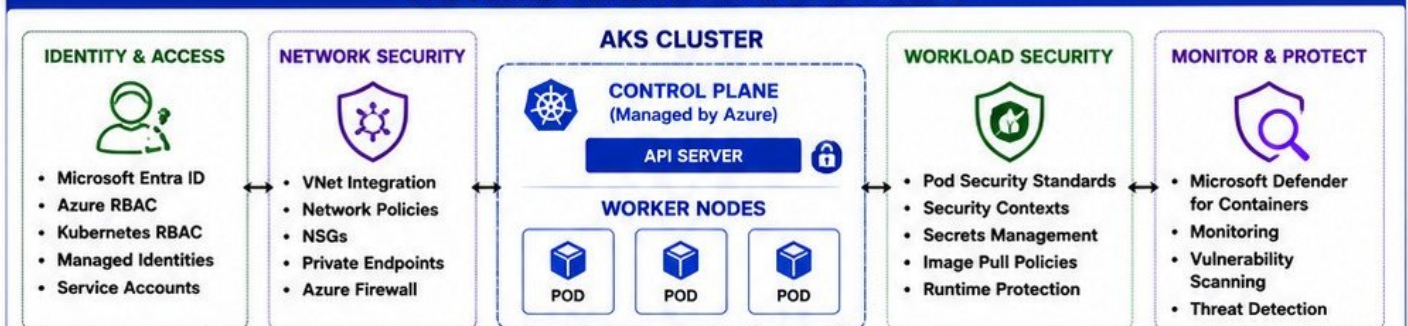
REPLICAS DO NOT PROVIDE HA IF EVERY REPLICA SHARES THE SAME FAILURE DOMAIN.
Spread your Pods across nodes and zones. Design for failure, not for luck.

13. AKS SECURITY HARDENING

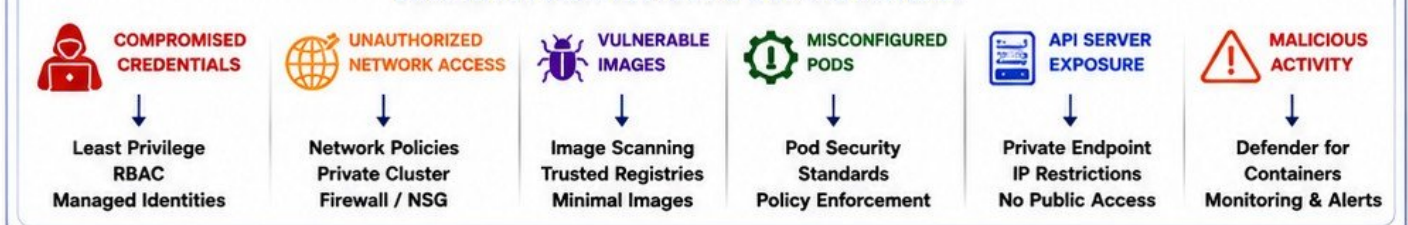
SECURE YOUR CLUSTER. PROTECT YOUR WORKLOADS. REDUCE RISK.

 1. LEAST PRIVILEGE Grant only the minimum permissions required for users, services, and pods. Review and remove excess access.	 2. KUBERNETES RBAC Use Roles and RoleBindings to control access to Kubernetes resources at the namespace or cluster level.	 3. AZURE RBAC Control access to AKS and Azure resources using Azure RBAC. Use Managed Identities.	 4. NETWORK POLICIES Restrict traffic between pods and namespaces. Allow only required communication.	 5. POD SECURITY CONTROLS Use Pod Security Standards or Azure Policy to enforce restricted security contexts for pods.
 6. IMAGE SECURITY Scan images for vulnerabilities. Use trusted registries. Keep base images updated and minimal.	 7. PRIVATE CLUSTERS Enable private cluster mode to remove public access to the API server. Use Private Link.	 8. API SERVER ACCESS Restrict API server access to specific IP ranges or VNets. Disable public network access.	 9. DEFENDER FOR CONTAINERS Enable Microsoft Defender for Containers for threat protection, posture management and runtime security.	 10. SECURITY BOUNDARIES Separate cluster components and workloads across network, identity, and resource boundaries.

AKS SECURITY ARCHITECTURE & BOUNDARIES



COMMON ATTACK VECTORS & HOW WE MITIGATE



SECURITY REMINDER

TREAT THE KUBERNETES API SERVER AS A PRIVILEGED PRODUCTION ENDPOINT. PROTECT IT AT ALL COSTS.

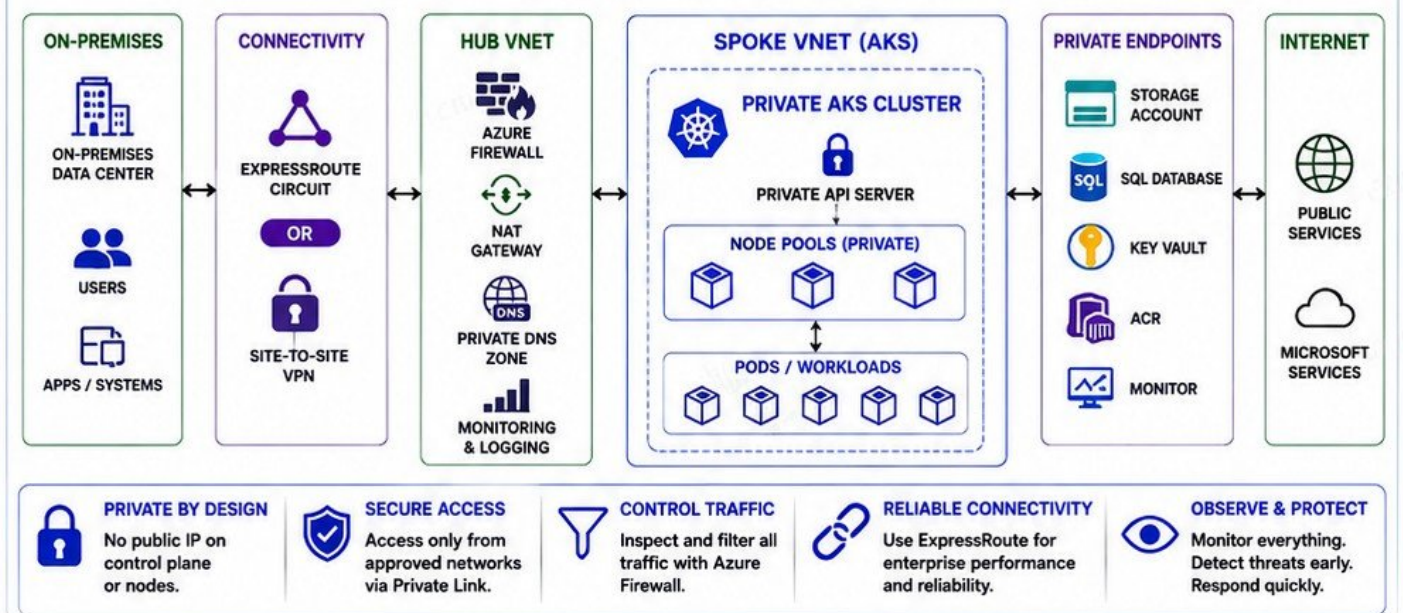
- ✓ Limit access
- ✓ Monitor continuously
- ✓ Apply defense in depth
- ✓ Assume breach and prepare

14. PRIVATE AKS AND ENTERPRISE NETWORK DESIGN

BUILD A PRIVATE, SECURE, AND CONNECTED AKS PLATFORM

 1. PRIVATE CLUSTERS Deploy AKS with private control plane. No public IP on the API server.	 2. PRIVATE API SERVER ACCESS Access the API server only from allowed networks using Private Link.	 3. PRIVATE ENDPOINTS Use Private Endpoints to access Azure PaaS services securely over the backbone.	 4. HUB-AND-SPOKE ARCHITECTURE Centralize shared services in the Hub. Spokes host AKS workloads.	 5. AZURE FIREWALL Inspect and control traffic to and from AKS and other resources.
 6. NAT GATEWAY Provide controlled outbound internet access for AKS private nodes.	 7. PRIVATE DNS Use Private DNS Zones for AKS private link records and name resolution.	 8. ON-PREMISES CONNECTIVITY Connect to on-premises networks securely via ExpressRoute or VPN.	 9. EXPRESSROUTE AND VPN Use ExpressRoute for high performance. Use VPN for branch or small offices.	 10. ENTERPRISE CONSIDERATIONS Plan for security, compliance, resilience, monitoring, and operational control.

ENTERPRISE ARCHITECTURE DIAGRAM (EXAMPLE)



ENTERPRISE ADVICE

PRIVATE DOES NOT AUTOMATICALLY MEAN SECURE.
ENFORCE IDENTITY, NETWORK, AND WORKLOAD CONTROLS
CONSISTENTLY ACROSS ALL LAYERS.

- ✓ Apply least privilege everywhere
- ✓ Restrict API server access
- ✓ Use network policies and segmentation
- ✓ Encrypt in transit and at rest
- ✓ Continuously monitor and audit
- ✓ Test, review, and improve regularly

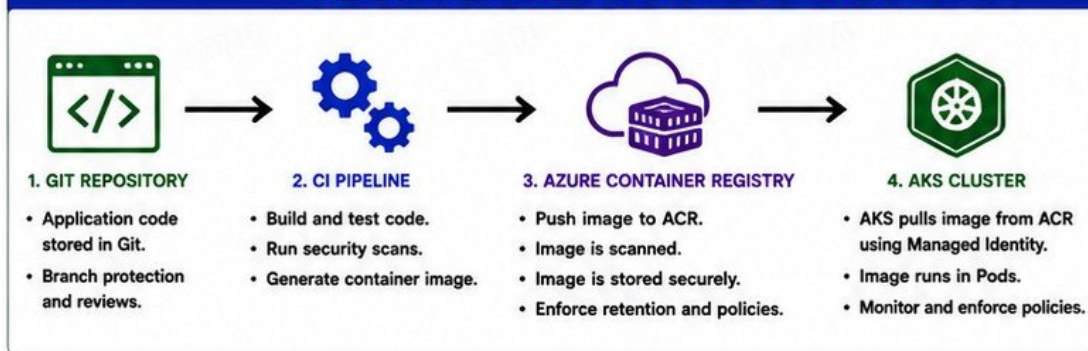


15. CONTAINER REGISTRY AND IMAGE SUPPLY CHAIN

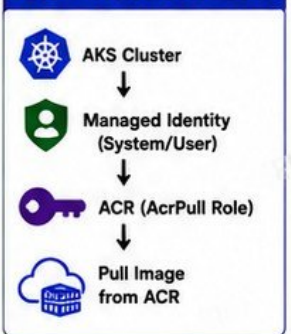
BUILD SECURE. STORE SECURE. DEPLOY SECURE.

 1. AZURE CONTAINER REGISTRY - Fully managed Docker registry by Microsoft. - Private, secure, scalable, and highly available.	 2. AKS-TO-ACR AUTHENTICATION - Use Managed Identity (recommended). - No credentials stored in Pods or nodes.	 3. MANAGED IDENTITIES - System-assigned or User-assigned MI. - Grant AcrPull role on the registry.	 4. IMAGEPULLSECRETS CONSIDERATIONS - Use only when Managed Identity is not possible. - Store creds securely in Kubernetes Secrets.	 5. IMAGE TAGGING - Use semantic versioning (e.g., 1.2.3). - Avoid "latest" in production.
 6. IMMUTABLE ARTIFACTS - Images should be immutable. - Never overwrite tags. Always create new.	 7. VULNERABILITY SCANNING - Enable ACR Tasks or Microsoft Defender for Container Registry. - Scan images on push and on schedule.	 8. IMAGE PROVENANCE - Know where images come from and how they were built. - Use attestations, signatures, and SBOMs.	 9. SUPPLY CHAIN SECURITY - Use trusted base images. - Restrict who can push images. - Enforce policies and sign images.	 10. BEST PRACTICES - Least privilege access to ACR. - Automate scanning. - Monitor and audit. - Keep base images updated.

DEPLOYMENT FLOW: GIT → CI → ACR → AKS



AKS → ACR AUTH FLOW



SECURITY CONTROLS THROUGHOUT THE SUPPLY CHAIN

 ACCESS CONTROL Use RBAC & Managed Identities	 VULNERABILITY SCANNING Scan images continuously	 POLICY ENFORCEMENT Enforce image policies in ACR and AKS	 SIGNING & ATTESTATION Sign images and verify provenance	 MONITORING & AUDITING Audit pulls, pushes, and changes
--	---	--	---	--

IMAGE TAGGING STRATEGY (EXAMPLES)

TYPE	EXAMPLE	USE CASE	RECOMMENDATION
SemVer	1.2.3	Stable releases	✓ RECOMMENDED
Build Number	1.2.3-build.45	Traceability	✓ RECOMMENDED
Commit SHA	a1b2c3d	Uniqueness	✓ RECOMMENDED
Latest	latest	Dev / Testing	✗ AVOID IN PROD



SECURITY REMINDER

NEVER TREAT A SUCCESSFUL IMAGE BUILD AS PROOF THAT THE IMAGE IS TRUSTWORTHY.

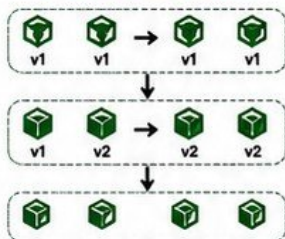
- ✓ Build success only means the image was created.
- ✓ It does NOT mean the image is secure.
- ✓ Malware, vulnerabilities, or misconfigurations can still exist.
- ✓ Always scan, verify, and enforce policies before deployment.

16. AKS DEPLOYMENT AND RELEASE STRATEGIES

DELIVER CHANGES SAFELY. MINIMIZE RISK. MAXIMIZE RELIABILITY.

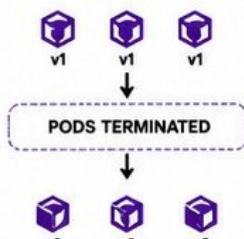
1. DEPLOYMENT STRATEGIES

ROLLINGUPDATE



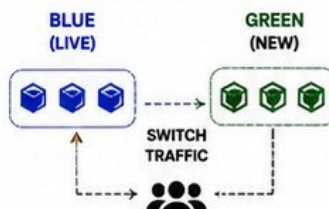
- Gradually replaces old Pods with new Pods.
- Default Kubernetes strategy.
- Zero-downtime when configured correctly.

RECREATE



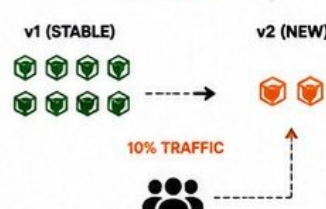
- Terminates all old Pods first.
- Then creates new Pods.
- Downtime expected.

BLUE-GREEN



- Deploy to Green environment.
- Test and validate.
- Switch traffic from Blue to Green.
- Rollback by switching back.

CANARY



- Release to a small subset of users.
- Monitor metrics and errors.
- Gradually increase traffic.
- Rollback quickly if issues occur.

2. PROBES – ENSURE YOUR APPLICATION IS REALLY READY

READINESS PROBE



Checks if a Pod is ready to accept traffic.

- If it fails, Pod is removed from Service endpoints.
- Prevents traffic from going to unready Pods.

LIVENESS PROBE



Checks if the application is alive.

- If it fails, Kubernetes restarts the Pod.
- Recovers from deadlocks or stuck processes.

STARTUP PROBE



Gives the application time to start.

- Disables liveness failures during startup.
- Prevents premature restarts.

3. DEPLOYMENT ROLLOUT



COMMIT & BUILD

Code is committed and built.



IMAGE PUSH

Container image pushed to registry.



DEPLOY NEW VERSION

Kubernetes creates new ReplicaSet.



ROLLING UPDATE / STRATEGY

Pods updated based on strategy.



VERIFY & MONITOR

Check metrics, logs, and user experience.

4. ROLLBACK STRATEGIES



ROLLBACK TO PREVIOUS REPLICASET

Use: `kubectl rollout undo deployment/<name>`



AUTOMATIC ROLLBACK

Use tools or Argo Rollouts for auto rollback on failed metrics.



KEEP OLD REVISIONS

Retain previous versions for quick rollback.



TEST ROLLBACK PATHS

Practice rollbacks before real incidents.

5. ZERO-DOWNTIME CONSIDERATIONS



USE ROLLING UPDATE, BLUE-GREEN, OR CANARY STRATEGIES



PROPER READINESS PROBES

Ensure traffic only goes to ready Pods.



ADEQUATE REPLICAS

Maintain sufficient capacity during rollout.



GRACEFUL SHUTDOWN

Configure terminationGracePeriodSeconds for clean Pod termination.



MONITOR EVERYTHING

Watch metrics, logs, and alerts in real time.

6. COMMON DEPLOYMENT ISSUES



PODS READY BUT USERS GET ERRORS
Application may not be fully initialized.



READINESS PROBE TOO PERMISSIVE
Traffic reaches Pod too early.



DEPENDENCY NOT READY

Database, cache, or external service not available.



RESOURCE LIMITS

CPU/Memory limits cause throttling or crashes.



TRAFFIC ROUTING ISSUES

DNS, Ingress, or Service misconfiguration.

PRODUCTION FAILURE: DEPLOYMENT REPORTS SUCCESS WHILE USERS RECEIVE ERRORS



COMMON ROOT CAUSES

- Readiness probe is incorrect
- Dependencies not ready
- Configuration mismatch
- Database migrations incomplete
- Cache not warmed up
- External APIs failing
- Insufficient replicas
- Traffic hitting old and new versions inconsistently



WHAT TO CHECK

- Pod status and events
- Readiness/Liveness probe logs
- Application logs
- Dependency health
- Database connectivity
- Ingress/Service routing
- Resource usage
- Metrics and user errors



IMMEDIATE ACTIONS

- Stop rollout if possible
- Rollback if impact is high
- Fix readiness probes
- Restore healthy dependencies
- Increase replicas if needed
- Communicate with users
- Monitor after mitigation



PREVENTION

- Test in staging like production
- Use proper probes
- Implement canary or blue-green
- Automate rollback
- Monitor continuously
- Have runbooks ready



SENIOR ENGINEER TIP

A successful deployment means Kubernetes did what you asked. It does NOT mean your users are having a good experience. Always verify.



SECURITY REMINDER

Scan images, enforce least privilege, and keep your supply chain trusted.

17. OBSERVABILITY AND PRODUCTION MONITORING

WHAT GOOD OBSERVABILITY DELIVERS



SEE WHAT IS HAPPENING



UNDERSTAND WHY IT HAPPENED



DETECT ISSUES EARLY



ALERT BEFORE USERS ARE IMPACTED



IMPROVE RELIABILITY CONTINUOUSLY

KEY OBSERVABILITY PILLARS



METRICS

System, Kubernetes and application metrics



LOGS

Structured and unstructured logs for debugging



EVENTS

Kubernetes and cluster events



TRACES (Optional)

Request flow across services



ALERTS

Proactive notifications for issues



SLI / SLOs

Measure user experience and reliability

AZURE OBSERVABILITY STACK FOR AKS

AZURE MONITOR



Central platform for metrics, logs, alerts and dashboards

CONTAINER INSIGHTS



AKS monitoring out of the box (metrics, logs, events)

MANAGED PROMETHEUS



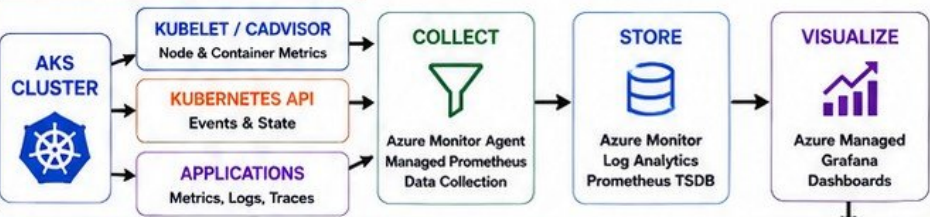
Scalable, high-availability metrics collection (Prometheus compatible)

AZURE MANAGED GRAFANA



Powerful dashboards and visualization at scale

OBSERVABILITY DATA FLOW



WHAT TO MONITOR (EXAMPLES)

	KUBERNETES HEALTH	Node status, Pod status, API server, Control plane, Scheduler, etc.
	CLUSTER RESOURCES	CPU, Memory, Disk, Network, PID, Filesystem, Node Pressure
	WORKLOADS	Pod restarts, CrashLoopBackOff, OOMKilled, Pending pods
	APPLICATIONS	Request rate, Error rate, Latency, Throughput, Dependencies
	PLATFORM SERVICES	DNS, Ingress, Load Balancer, Storage, ACR pulls, Key Vault
	BUSINESS / USER EXPERIENCE	Availability, Success rate, Checkout success, Page load time

SLI AND SLO THINKING

SLI (Service Level Indicator)

A measurable metric that reflects user experience.

Examples:

- HTTP 5xx rate
- Request latency (p95)
- Successful requests %
- Checkout success rate

SLO (Service Level Objective)

Target value for the SLI over a defined time window.

Examples:

- 99.9% availability
- p95 latency < 300ms
- Error rate < 0.1%
- 99% checkout success rate

ERROR BUDGET

The allowable amount of unreliability before users are impacted.



ALERT



Azure Monitor Alerts
Action Groups
Notifications



SENIOR ENGINEER TIP

MONITOR USER-VISIBLE FAILURE SIGNALS,
NOT MERELY CLUSTER HEALTH.

EXAMPLES OF USER-VISIBLE SIGNALS

- Users cannot log in
- Page loads are slow (high latency)
- Transactions fail
- Errors increase
- Users abandon flows



18. AKS TROUBLESHOOTING FRAMEWORK

A SYSTEMATIC APPROACH TO RESOLVE PRODUCTION ISSUES

ESSENTIAL KUBECTL COMMANDS



kubectl get

List resources and get a quick overview



kubectl describe

Detailed resource information, events, and conditions



kubectl logs

View container logs for errors and context



kubectl events

See real-time events across the cluster

WHAT TO INVESTIGATE



POD STATUS

Check phase, restarts, READY status, and container states.



NODE CONDITIONS

Check node status, capacity, pressure, taints, and readiness.



DNS INVESTIGATION

Verify service discovery, CoreDNS logs, and name resolution.



NETWORK INVESTIGATION

Check Services, Endpoints, Network Policies, Ingress, and routing.



RESOURCE PRESSURE

CPU, Memory, Disk pressure, requests/limits, throttling.



IDENTITY FAILURES

RBAC, Service Accounts, Azure AD, Workload Identity, token issues.



STORAGE FAILURES

PVC status, StorageClass, CSI driver, mounts, permissions, capacity.

TROUBLESHOOTING FLOW



1. SYMPTOM

What is failing?
What is the user impact?



2. SCOPE

Identify the affected component or boundary.



3. EVIDENCE

Collect facts: logs, events, metrics, states.



4. HYPOTHESIS

Form the most likely cause based on evidence.



5. TEST

Run safe tests to confirm or eliminate cause.



6. MITIGATE

Apply the fix or workaround to restore service.



7. VERIFY

Confirm the issue is resolved and no side effects.

COMMON FAILURE PATTERNS & QUICK CHECKS

POD WON'T START



- Check Events
- Check Image
- Check Resources
- Check Probes

POD CRASHING



- Check Logs
- Check OOMKilled
- Check Probes
- Check Config

POD PENDING



- Check Resources
- Check Node Selector
- Check Taints/Tolerations
- Check PVC binding

SERVICE NOT REACHABLE



- Check Endpoints
- Check Selector
- Check Network Policy
- Check DNS

PERFORMANCE ISSUE



- Check Metrics
- Check CPU/Memory
- Check Throttling
- Check Dependencies



DEBUGGING INSIGHT

START WITH THE FAILURE BOUNDARY BEFORE CHANGING ANYTHING.

19. AKS UPGRADES AND DAY-2 OPERATIONS

KEEP YOUR CLUSTER SECURE, RELIABLE AND PRODUCTION READY

1. KUBERNETES VERSION LIFECYCLE



- New Release**
New Kubernetes version is released.
- Active Support**
Regular updates and security fixes.
- Maintenance**
Critical fixes only.
- End of Life (EOL)**
No fixes. High risk for production.
- Plan Ahead**
Upgrade before EOL to stay secure and supported.



2. UPGRADE TYPES



CONTROL-PLANE UPGRADES

Upgrades the managed control plane. Azure handles HA and failover.



NODE IMAGE UPGRADES

Updates the OS and runtime on nodes. New images = bug fixes + security.



NODE POOL UPGRADES

Upgrade Kubernetes version for the node pool.



SURGE UPGRADES

Add new nodes first, then drain old nodes. Minimizes downtime.



PODDISRUPTIONBUDGETS (PDB)

Protect critical workloads from being disrupted during upgrades.

3. DAY-2 OPERATIONS ESSENTIALS



MAINTENANCE WINDOWS

Schedule upgrades during low-traffic periods.



UPGRADE TESTING

Test in non-production first. Validate apps and dependencies.



BACKUP & RECOVERY

Back up etcd (automatic for AKS). Back up application data and configs.



MONITOR & ALERT

Monitor cluster, nodes and workloads before, during and after upgrades.



DOCUMENT & AUTOMATE

Use IaC and runbooks for repeatable and consistent upgrades.

4. PRODUCTION UPGRADE WORKFLOW

1



PLAN

Check version support, release notes and breaking changes.

2



ASSESS

Review cluster and workloads. Check PDBs, quotas and dependencies.

3



TEST

Test upgrade in staging. Validate apps, integrations and scale.

4



BACKUP

Confirm etcd backup and application backup are available.

5



UPGRADE

Perform control-plane upgrade then node pool or node image upgrade.

6



VALIDATE

Verify cluster health, app health, metrics and logs.

7



MONITOR

Monitor closely for issues. Watch for errors, latency and capacity.

5. UPGRADE BEST PRACTICES



STAY ON SUPPORTED VERSIONS



USE SURGE UPGRADES WHEN POSSIBLE



CONFIGURE PDBs FOR CRITICAL APPS



VALIDATE IN NON-PROD FIRST



MONITOR BEFORE, DURING AND AFTER



AUTOMATE AND DOCUMENT YOUR PROCESS



PRODUCTION NOTE:

AN UPGRADE STRATEGY IS ALSO A WORKLOAD AVAILABILITY STRATEGY.

20. PRODUCTION AKS REFERENCE ARCHITECTURE

END-TO-END PRODUCTION AKS ARCHITECTURE

